

ElvisPOS Remote Terminal — Installation and Operations Guide

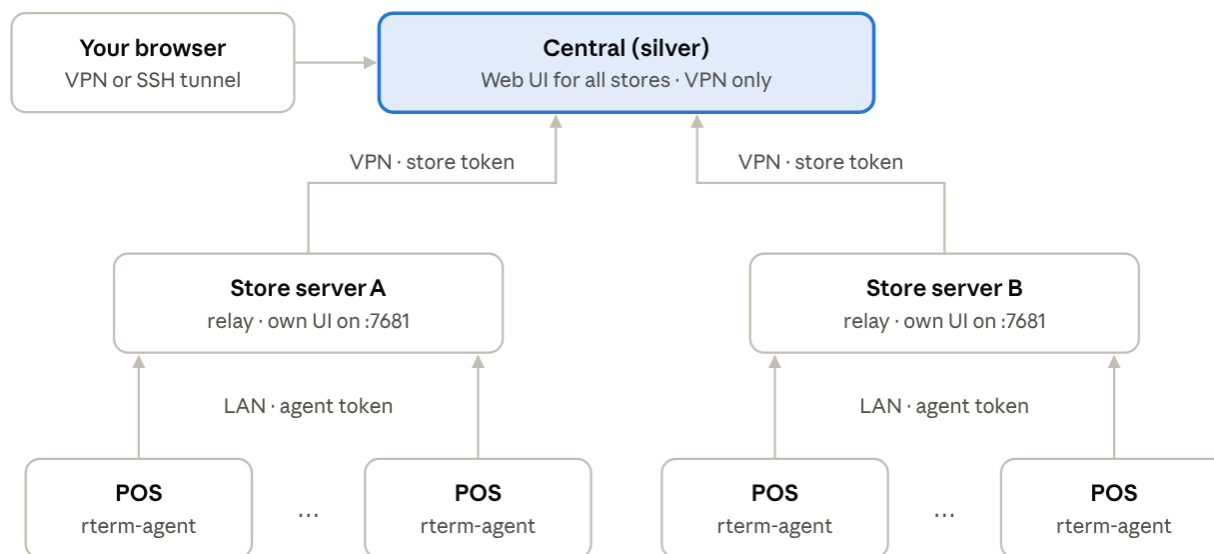
Oct 2, 2026 · @Lito

Overview

ElvisPOS Remote Terminal gives you browser access to every POS in the fleet from the central server. From one page you get a shell, the POS screen over VNC, log downloads and fleet-wide commands. Every connection is opened outward: each POS dials its store server, and each store server dials central over the VPN. Nothing ever connects into a store or a POS.

- **Central** runs the web UI that you use. It lists every store and, under each store, its POS.
- **Store servers** run the same server in relay mode. Their own POS connect to them, and the store forwards everything up to central over a single link.
- **POS** run a small agent, `rterm-agent`. It holds one connection to its store and opens a shell, a VNC bridge or a log archive only when someone asks for one.

Every link is opened from below: POS to store, store to central



Arrows show who opens each connection on port 7681; data then flows both ways.

topology · central, store servers, POS

Central never connects to a store, and a store never connects to a POS. A store or POS behind NAT or a firewall works as long as it can reach the tier above it on port 7681.

Each store server also serves its own UI on port 7681, which shows only that store's POS. This is useful when central is unreachable.

Components

Each tier runs a single piece. `ecli rterm install` detects the role automatically: a machine without Docker is a POS, a server with `storeReplicationID` in `ecli.json` is a store, and anything else is central.

Where	What runs	Files and settings
Central	Docker Swarm stack <code>rterm</code> , service <code>rterm_agent</code> , image <code>registry.elvispos.com/com-elvispos-rterm-server:latest</code> , port 7681 (host mode)	<code>~/com-elvispos-engine/docker-compose.rterm-agent.yml</code> ; <code>RTERM_*</code> keys in <code>~/com-elvispos-engine/config/elvisenv</code> ; volume <code>rterm_rterm-data</code> (list of known POS); POS release in <code>share/RELEASE/elvispos/rterm/</code> and <code>share/ALL/elvispos/rterm/</code>
Store server	The same stack and image, linked to central	The same compose file, <code>elvisenv</code> keys and volume; POS token in <code>share/STORE_SEEDS/rterm-agent.env</code> ; POS release copied from central
POS	<code>rterm-agent</code> , a detached process (no daemon), running as <code>elvispos</code>	<code>/usr/share/elvispos/rterm/</code> : <code>rterm-agent</code> , <code>start</code> , <code>stop</code> , <code>rterm.env</code> , <code>rterm.pid</code> , <code>start.log</code> , <code>log/rterm-agent.log</code> ; a line in root's crontab starts it at boot

The server container is attached to `network-backend`, so it can reach `ybservice` at `http://ybservice:7392`. It uses `ybservice` to look up store and device names. On the servers, `ecli start|stop|restart|logs rterm` manage the stack like any other ElvisPOS service.

How it works

Each POS and each store server keeps one WebSocket **control link** open to the tier above it. Every terminal, screen or log download then gets its own **session link**, which the lower tier opens on request.

Control links

- A POS connects to `ws://memphisserver:7681` with its store's agent token. It announces its id (hostname, or `--id`), its device id (`ELVIS_DEVICE_ID` from `/usr/share/elvispos/elvisenv`), its IP, uptime and version. If the link drops it reconnects with a 1-30 s backoff, and the `start` loop restarts the agent 5 s after it exits.
- A store server connects to central at `RTERM_UPSTREAM` with central's store token. It sends its store id and an up-to-date list of its POS. In central, POS are keyed `<store id>/<pos id>`.

Opening a terminal, screen or log download

1. The browser opens a WebSocket to central (same origin, logged-in cookie).
2. Central sends an `open` request down the control link to the store, and the store passes it to the POS.
3. The POS dials back a new session link to the store, carrying a one-time session id. The store does the same to central.
4. Each server splices the two sockets together, so keystrokes, screen data or the log archive flow straight between the browser and the POS.

The server is never more than a pipe. A shell is a PTY running as `elvispos`. A screen session is a TCP bridge to the POS's own `x11vnc` on `127.0.0.1:5900`. A log download is a `.tar.gz` streamed from `/usr/share/elvispos`.

Commands across POS. *Run a command* sends `sh -c <command>` to each selected POS over its control link and collects exit code and output, with a timeout. A result is accepted only from the link the request went out on.

Names. Each server asks ybservice for `system.store` (store id → description) and `system.devices` (device id → description) through its free-query API. It refreshes every 5 minutes, and immediately when an unknown id appears. The sidebar shows these descriptions, with hostname and IP underneath.

Release. The POS agent ships inside the server image. Central extracts it into its shares, store servers copy it from central, and POS copy it from their store. See *Releasing a new version*.

Prerequisites

Check each item before installing. Install in order: central first, then each store, then that store's POS.

Need	Where	Why
An <code>ecli</code> build that has the <code>ecli rterm</code> command	Central, stores, POS	Every install, upgrade and removal goes through it
Image <code>com-elvispos-rterm-server:latest</code> in <code>registry.elvispos.com</code>	Registry	The servers pull it, and the POS agent is extracted from it
Docker Swarm active, <code>network-backend</code> and <code>ybservice</code> running	Central, stores	The stack joins the ElvisPOS backend network to read names
VPN between stores and central	Stores → central	Stores dial <code>ws://<central VPN IP>:7681</code> , and central accepts only VPN addresses
Port 7681/tcp reachable	Central from stores (VPN); store from its POS (LAN)	The only port the system uses
<code>ecli</code> SSH link from the store to central (<code>sshParms</code> in <code>ecli.json</code>)	Stores	Store install reads central's store token and copies the POS release over it
<code>storeReplicationID</code> in <code>ecli.json</code>	Stores	Becomes the store id shown on central
<code>rsync</code> access to <code>memphisserver::share</code>	POS	The POS gets its agent, scripts and token from its store
<code>elvispos</code> account with <code>sudo</code> ; <code>x11vnc</code> on <code>127.0.0.1:5900</code> (optional, for <code>Screen</code>)	POS	The agent and its shells run as <code>elvispos</code> ; <code>sudo</code> is used only to write root's crontab

Install 1: central server

Installing central is one command, and running it again later is safe:

```
ecli rterm install
```

It does the following:

1. Writes the `RTERM_*` keys into `config/elvisenv`. Values that already exist are kept, so secrets survive re-runs. It generates the agent token, the admin password (user `admin`), the session secret and the **store token**.
2. Sets `RTERM_ALLOW_CIDRS` to the VPN subnets of this host's `tun*` / `wg*` interfaces, plus `127.0.0.0/8` and the `docker_gwbridge` gateway (for example `7.7.7.0/24,127.0.0.0/8,172.18.0.1/32`). Any other address gets `403`. To override it, use `--allow-cidrs`.
3. Writes `docker-compose.rterm-agent.yml`, pulls the image, deploys the stack `rterm` and waits for it to run.
4. Publishes the POS release. It extracts `rterm-agent` from the image into `share/RELEASE/elvispos/rterm/` and writes `start` / `stop` into `share/ALL/elvispos/rterm/`.

When it finishes, open `http://<central VPN IP>:7681` from the VPN and log in as `admin`. The password is `RTERM_ADMIN_PASSWORD` in `~/com-elvispos-engine/config/elvisenv`. From outside the VPN, use an SSH tunnel and open `http://localhost:7681`:

```
ssh -p 4168 -L 7681:localhost:7681 elvispos@<central host>
```

Options: `--tag vX.Y.Z` pins an image version instead of `latest`.

Install 2: each store server

On each store server, run the same command after central is installed:

```
ecli rterm install
```

It does the following:

1. Sets `RTERM_STORE_ID` from `storeReplicationID` in `ecli.json`.
2. Finds central's address, in this order:
 - a. `--central <ip>`, if given;
 - b. `sshParms.host` from `ecli.json`, resolved to an IP and used if it falls inside one of this host's VPN subnets. A store that is repointed to another central follows it on the next install;
 - c. the `RTERM_UPSTREAM` already saved (for example an IP typed at an earlier prompt);
 - d. otherwise it asks *Central server VPN IP:*. Without a terminal it stops and asks for `--central`.

It saves the IP, not the hostname, because containers do not see `/etc/hosts`. The result is `RTERM_UPSTREAM=ws://<ip>:7681`.

3. Reads central's store token over the ecl SSH link into `RTERM_UPSTREAM_TOKEN`. You can pass it yourself with `--store-token`.
4. Generates this store's own agent token, admin password and session secret, and deploys the stack.
5. Waits until central confirms the link (*Linked to central*), then copies the POS release from central into its own `share/RELEASE` and `share/ALL`.
6. Publishes the POS token in `share/STORE_SEEDS/rterm-agent.env`, where the store's POS fetch it.

The store now appears in central's **Stores** card. Each store has its own agent token, so a POS token from one store does not work on another. `config/elvisenv` and `share/STORE_SEEDS` are not synced between servers.

Install 3: each POS

On each POS, as `elvispos`, run the same command again:

```
ecli rterm install          # name in the UI = hostname
ecli rterm install --id cassa3 # or a chosen name
```

It does the following:

1. Fetches the agent token from `memphisserver::share/STORE_SEEDS/rterm-agent.env` by rsync. The token is never printed or logged.
2. Fetches `rterm-agent` from `share/RELEASE/elvispos/rterm/` and `start / stop` from `share/ALL/elvispos/rterm/` into `/usr/share/elvispos/rterm/`.
3. Writes `rterm.env` (mode 600) with `RTERM_SERVER=ws://memphisserver:7681`, `RTERM_TOKEN`, and `RTERM_ID` if `--id` was given.
4. Adds this line to root's crontab, next to `startvnc` and `startall`:

```
@reboot su elvispos -c /usr/share/elvispos/rterm/start >>
/usr/share/elvispos/rterm/start.log 2>&1
```

5. Restarts the agent in the background and waits for its pid. It is safe to run this from a Remote Terminal shell on that same POS: the shell drops for a moment, and the POS reconnects.

`start` runs the agent in its own session (`setsid`) inside a restart loop and records the loop's pid in `rterm.pid`. It is not run in `screen`, because the POS `startall` runs `killall -9 screen` at every boot. `stop` ends the whole process group. The agent log is

`log/rterm-agent.log` (mode 600, directory 700), and the previous run's log is kept as `rterm-agent.log.1`.

Releasing a new version

A new version needs one image build. Servers pick it up with `ecli rterm install`, and POS pick it up through the normal ElvisPOS release sync. `ecli` never needs a rebuild for it, because it always deploys `latest`.

1. **Build and push** from `~/dev/remote-terminal`:

```
make push VERSION=vX.Y.Z
```

This pushes both `:vX.Y.Z` and `:latest`. The image contains the server, the web UI and the POS agent.

2. **Central:** run `ecli rterm install`. It redeploys the stack and republishes the agent and scripts into `share/RELEASE` and `share/ALL`. To refresh only the POS release without touching the stack, run `ecli rterm publish`.
3. **Store servers:** run `ecli rterm install` to update the server and copy the new agent from central right away. Otherwise the agent reaches the store with the regular share sync from central.
4. **POS:** an agent update is a normal POS release. POS copy `elvispos/rterm/` from their store when `swdist.device_release` triggers an update, or at once with `ecli sync --force`. The new binary runs from the next agent restart or reboot. To switch immediately, run `ecli rterm install` on the POS, or `/usr/share/elvispos/rterm/stop` followed by `start`.

To see which agent version a POS runs, hover over the POS in the sidebar. The tooltip shows `agent vX.Y.Z`.

Using the web UI

The UI has a POS list on the left and tabs on the right. Click a POS to open a terminal. Hover over a POS to get its **Screen** and **Logs** buttons.

Login and language. Sign in as `admin`. The language selector (English or Italiano, English by default) is on the login page and at the top of the sidebar. The browser remembers the choice, and a login lasts 12 hours.

Sidebar

- Counters show POS online, POS known and stores up. The search box matches store name, POS name, device id, hostname and IP. Use *All / Online / Offline* to filter.

- **Sort:** **A-Z** orders stores and POS by name. **Store ID** orders them by number and shows each store id in blue.
- POS are grouped under their store's `system.store` description. Each row shows the device's `system.devices` description, with hostname · IP · uptime underneath and a badge counting its open sessions. Hover for details (device id, user, OS, agent version, address).
- *Remove from list* drops an offline POS or store. It comes back if it connects again.

Terminals and split panes

- Each tab shows 1, 2 or 4 panes. Use the layout buttons at the top right. Clicking a POS fills the focused pane, or switches to that POS if it is already open in the tab.
- **Type in all** (amber when on) sends your typing to every terminal pane of the tab, for running the same thing on up to 4 POS side by side.
- If a shell drops, press Enter in the pane to reconnect.

Terminal | Screen. Every pane header has a *TERMINAL / SCREEN* switch. It flips the same POS between its shell and its screen, and the other view stays connected in the background.

Screen (VNC)

- The UI first checks that VNC is running on the POS. If it is not, it shows *VNC is not running on this POS*.
- Type the POS's VNC password in the browser. It is used only for that view and is never stored on the servers.
- The screen opens **view only**. *Take control* passes your mouse and keyboard to the POS, and *Ctrl+Alt+Del* is available only while you are in control.

Logs. The *Logs* button opens a tab listing the log files under `/usr/share/elvispos` with size and age. These are `*.log*` files, or files in `log / logs` folders, up to 3 levels deep. Use *Download selected* or *Download all*. The POS compresses the files and you receive one `.tar.gz`.

Fleet tab

- *Run a command:* tick POS (or *Select shown*, or a whole store), type a command, choose a timeout and run it. Each POS reports its exit code and output.
- *Stores:* each store with its uptime, POS online/total and address.
- *Add a store* (on central) and *Add a POS* repeat the install commands, with copy buttons.

Operations

All day-to-day tasks go through `ecli`. Run them on the machine concerned.

Task	Command	Where
Check state	<code>ecli rterm status</code>	Servers: role, store id, upstream, allowed networks, service state. POS: running pid, boot entry, last log lines
Start / stop / restart the server	<code>ecli start rterm</code> , <code>ecli stop rterm</code> , <code>ecli restart rterm</code>	Central, stores
Server log and audit trail	<code>ecli logs rterm</code> or <code>docker service logs rterm_agent</code>	Central, stores
Start / stop the agent	<code>/usr/share/elvispos/rterm/start</code> , <code>.../stop</code> (as <code>elvispos</code>)	POS
Agent log	<code>/usr/share/elvispos/rterm/log/rterm-agent.log</code> ; boot output in <code>start.log</code>	POS
Refresh the POS release only	<code>ecli rterm publish</code>	Central
Remove	<code>ecli rterm uninstall</code> (<code>-f</code> skips the confirmation)	Any
Remove with settings and data	<code>ecli rterm uninstall --purge</code>	Central, stores

On a POS, `uninstall` stops the agent, removes the crontab line and deletes `/usr/share/elvispos/rterm`. On a server, it removes the stack. `--purge` also deletes the `RTERM_*` keys from `elvisenv` and the `rterm_rterm-data` volume.

Rotating secrets. Delete the key from `config/elvisenv` and run `ecli rterm install` again. A new value is generated.

- `RTERM_ADMIN_PASSWORD`: takes effect immediately.
- `RTERM_AGENT_TOKEN` on a store: re-run `ecli rterm install` on each of that store's POS.
- `RTERM_STORE_TOKEN` on central: re-run `ecli rterm install` on every store.

Settings. These are the keys in `~/com-elvispos-engine/config/elvisenv`, all written by `ecli rterm install`:

Key	Tier	Meaning
<code>RTERM_ADMIN_USER</code> , <code>RTERM_ADMIN_PASSWORD</code>	Central, stores	Web UI login (user <code>admin</code>)
<code>RTERM_SESSION_SECRET</code>	Central, stores	Signs login cookies
<code>RTERM_AGENT_TOKEN</code>	Central, stores	Secret this server's POS present (published to POS via <code>share/STORE_SEEDS</code>)
<code>RTERM_YBSERVICE_URL</code>	Central, stores	<code>http://ybservice:7392</code> , source of store and device names
<code>RTERM_STORE_TOKEN</code>	Central	Secret store servers present when they link
<code>RTERM_ALLOW_CIDRS</code>	Central (optional on stores)	Networks allowed to connect; others get 403
<code>RTERM_STORE_ID</code>	Store	<code>storeReplicationID</code> from <code>ecli.json</code>
<code>RTERM_UPSTREAM</code>	Store	<code>ws://<central VPN IP>:7681</code>
<code>RTERM_UPSTREAM_TOKEN</code>	Store	Central's <code>RTERM_STORE_TOKEN</code>
<code>RTERM_PUBLIC_URL</code>	Optional	Address shown in <i>Add a store</i> , only behind a reverse proxy

On the POS, `/usr/share/elvispos/rterm/rterm.env` holds `RTERM_SERVER`, `RTERM_TOKEN` and the optional `RTERM_ID`.

Security

Access is gated three ways: central is reachable only from the VPN, each tier has its own secret, and nothing on a store or POS accepts inbound connections from central.

- **Network.** Central answers only addresses in `RTERM_ALLOW_CIDRS` (VPN, localhost, Docker gateway). From outside, use the SSH tunnel. Traffic is plain `ws / http` and relies on the VPN for encryption.

- **Three separate secrets.** The admin login (per server), the POS agent token (per store) and central's store token. A leaked POS token gives no access to the UI or to other stores.
- **Browser.** Login cookies are HMAC-signed and expire after 12 hours. WebSocket requests from another origin are rejected.
- **POS.** Shells run as `elvispos`, not root. `rterm.env` and the agent log are readable only by `elvispos`. Log downloads are limited to regular files under `/usr/share/elvispos` (no symlinks). The VNC password never leaves the browser and the POS.
- **Audit.** Each server logs logins, commands run, terminal, screen and log-download sessions with who and which POS. Read them with `docker service logs rterm_agent`.

Troubleshooting

Symptom	Cause	Fix
Central page returns <code>403 Forbidden</code>	Your address is not in <code>RTERM_ALLOW_CIDRS</code>	Connect to the VPN or use the SSH tunnel; extend with <code>ecli rterm install --allow-cidrs ...</code>
Store install asks <i>Central server VPN IP:</i>	<code>ecli.json</code> host is a public name or not on the VPN	Type central's VPN IP, or pass <code>--central <ip></code>
Store install: <i>Not linked to central yet</i>	Port 7681 blocked on the VPN, wrong IP or token	<code>ecli rterm status</code> and <code>ecli logs rterm</code> on the store; re-run <code>ecli rterm install</code> on the store
POS install: <i>Agent token not found</i>	Store not installed, or <code>rsync</code> to <code>memphisserver</code> fails	Install the store first; check <code>rsync elvispos@memphisserver::share/</code> from the POS
POS install: <i>rterm is not published on memphisserver yet</i>	The store has no POS release	Run <code>ecli rterm install</code> on central, then on the store
POS offline after reboot	Crontab line missing, or agent cannot reach the store	<code>ecli rterm status</code> on the POS; read <code>start.log</code> and <code>log/rterm-agent.log</code>
Two POS overwrite each other in the list	Same hostname in the same store	<code>ecli rterm install --id <unique name></code> on one of them

Symptom	Cause	Fix
Screen: <i>VNC is not running on this POS</i>	x11vnc not listening on <code>127.0.0.1:5900</code>	Start VNC on the POS (<code>startvnc</code>)
Screen: <i>Wrong VNC password</i>	Password typed does not match the POS	Use that POS's VNC password
Names show ids instead of descriptions	ybservice unreachable or the id is missing from <code>system.store</code> / <code>system.devices</code>	Check that ybservice runs; names refresh within 5 minutes